VoiceShield AI - Transferable Model Package (SIH PS-26104)

This folder contains the complete, portable, framework-agnostic export of the **Enterprise Voice Conformer** model trained from scratch on real human speech and deepfake voice clone datasets.

■ Package Contents

File	Format	Description
`voiceshield_conformer.onnx`	**ONNX Format (625 KB)**	Universal, framework-agnostic model. Runs on CPU/GPU without needing PyTorch.
`voiceshield_conformer.pt`	**TorchScript (670 KB)**	Self-contained traced PyTorch model. Loadable directly via `torch.jit.load()`.
`voiceshield_weights.pth`	**PyTorch State Dict (583 KB)**	Raw PyTorch model weights dictionary.
`voiceshield_client.py`	**Python SDK / Wrapper**	Self-contained client with built-in 32-dim feature extraction and VAD preprocessing.
`demo_friend_integration.py`	**Integration Demo Script**	Ready-to-run demonstration showing 3-line integration and model ensembling.

■ Quick Setup on Friend's Laptop

Your friend does **NOT** need to install your entire project repository. They only need this `model_export/` directory!

Option A: Lightweight ONNX Runtime (RECOMMENDED - No PyTorch required!)

```
pip install onnxruntime soundfile numpy scipy
```

Option B: PyTorch Environment

```
pip install torch soundfile numpy scipy
```

■ 3-Line Integration Example

```
from voiceshield_client import VoiceCloneDetector

# 1. Initialize detector (choose 'onnx' or 'torch')
detector = VoiceCloneDetector(model_format="onnx")
```

```
# 2. Analyze any audio file (.wav, .flac, .ogg, or 1D numpy array)
result = detector.predict("incoming_call.wav")

# 3. Access forensic predictions
print(result["prediction"])        # "REAL" or "FAKE"
print(result["fake_probability"])  # Float between 0.0 and 1.0 (e.g. 0.9984)
print(result["confidence_pct"])    # Percentage confidence (e.g. 99.84%)
print(result["risk_level"])        # "LOW", "SUSPICIOUS", or "HIGH"
print(result["badge"])             # "■ LOW RISK" or "■ HIGH RISK (ATTACK)"
```

■ Ensembling with Another Model

If your friend has their own deep learning or NLP model (e.g. sentiment, transcription, or another acoustic model), you can easily fuse the probabilities:

```
from voiceshield_client import VoiceCloneDetector

# Initialize VoiceShield
detector = VoiceCloneDetector(model_format="onnx")

# 1. Get VoiceShield fake probability
vs_result = detector.predict("audio_call.wav")
p_voiceshield = vs_result["fake_probability"]

# 2. Get Friend's model fake probability
p_friend = your_friend_model.predict("audio_call.wav")

# 3. Weighted Ensemble (50% VoiceShield + 50% Friend's Model)
ensemble_score = (0.50 * p_voiceshield) + (0.50 * p_friend)

if ensemble_score >= 0.70:
    action = "■ CRITICAL ATTACK - BLOCK CALL & FREEZE TRANSACTION"
elif ensemble_score >= 0.40:
    action = "■ SUSPICIOUS - TRIGGER SECONDARY OUT-OF-BAND AUTH"
else:
    action = "■ AUTHENTIC - PASS"

print(f"Combined Threat Score: {ensemble_score:.4f} -> {action}")
```

■ Verification & Testing

To test the package immediately on the friend's laptop, run:

```
python demo_friend_integration.py
```

Expected output:

```
[1] Initializing VoiceCloneDetector (Backend: ONNX Runtime)...
    Detector loaded successfully!

[2] Testing Real Human Sample:
    • Verdict:          ■ LOW RISK (AUTHENTIC)
    • Fake Probability: 0.12%

[3] Testing Deepfake Clone Sample:
    • Verdict:          ■ HIGH RISK (ATTACK)
```

- Fake Probability: 99.84%